Tight Lower Bounds for Weighted Matroid Problems

Ilan Doron-Arad* Ariel Kulik† Hadas Shachnai‡

July 1, 2023

1 Introduction

2 Preliminaries

2.1 Notations

For the simplicity of the notation, for any set A and some element a, let $A + a$, $A - a$ be $A \cup \{a\}$ and $A \setminus \{a\}$, respectively. In addition, for all $k \in \mathbb{N}$ let $[k]$ be the set $\{1, 2, \ldots, k\}$. For any function $f : A \to B$ and $C \subseteq A$ let $f(C) = \sum_{\ell \in C} f(\ell)$. Finally, given the set of numbers $[n]$ for some $n \in \mathbb{N}$ and $S \subseteq [n]$, we use $\mathsf{sum}(S) = \sum_{i \in S} i$. We use $\mathrm{OPT}(I)$ to denote the optimum value of an instance I of some optimization problem.

2.2 Randomized Algorithms and Approximation Schemes

Our hardness results hold also for randomized algorithms. In this context, a *randomized polynomial* algorithm $\mathcal{A}$ for a decision problem D, is an algorithm that given an instance I of D runs in time $|I|^{O(1)}$ and returns the following.

- If I is a no-instance, the algorithm returns it is a no-instance with probability (w.p.) 1.
- If I is a yes-instance, the algorithm returns that I is a yes-instance w.p. at least $\frac{1}{2}$.

We consider optimization problems that do not necessarily have a solution; however, in all of these problems deciding if a solution exists is polynomially solvable. For an instance I of an optimization problem P, let $\mathrm{OPT}(I)$ be the optimum value of a solution for I. For some $\rho \geq 1$, a randomized ρ-approximation algorithm $\mathcal{A}$ for an optimization problem P, is an algorithm that given an instance I of P runs in time $|I|^{O(1)}$ and returns the following.

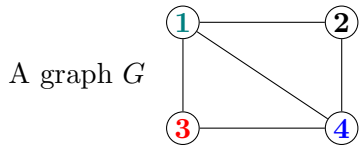
- If I does not have a solution, the algorithm returns that I does not have a solution w.p. 1.
- If I has a solution, the algorithm returns w.p. at least $\frac{1}{2}$ a solution S for I of value satisfying:
  - At least $\frac{\mathrm{OPT}(I)}{\rho}$ if P is a maximization problem.
  - At most $\rho \cdot \mathrm{OPT}(I)$ if P is a minimization problem.

For $\rho = 1$, we say that $\mathcal{A}$ is a randomized exact algorithm for P. For both decision and optimization problems, if the second condition, respectively holds w.p. 1, we say that algorithm $\mathcal{A}$ is deterministic.

*Computer Science Department, Technion, Haifa, Israel. idoron-arad@cs.technion.ac.il

†CISPA Helmholtz Center for Information Security, Saarland Informatics Campus, Germany. ariel.kulik@cispa.de

‡Computer Science Department, Technion, Haifa, Israel. hadas@cs.technion.ac.il



$\mathcal{J}_{4,2} = \{\emptyset, \{\mathbf{1}\}, \{\mathbf{2}\}, \{\mathbf{3}\}, \{\mathbf{4}\}\}$, $\mathcal{K}_{4,2,5} = \{\{\mathbf{2},\mathbf{4}\}, \{\mathbf{2},\mathbf{1}\}, \{\mathbf{4},\mathbf{3}\}, \{\mathbf{3},\mathbf{1}\}\}$, $\mathcal{L}_{4,2,5}(\Pi) = \{\{\mathbf{2},\mathbf{3}\}\}$

Figure 1: The independent sets of the Π-matroid $M_{n,k,\alpha}(\Pi)$, with parameters $n = 4$, $k = 2$, $\alpha = 5$. The secret set Π contains all independent sets in the graph G, where $\{2, 3\}$ is the only independent set in G with k elements.

For an instance I of a combinatorial optimization problem P, let $|I|$ be the encoding size of I. A (randomized) *polynomial-time approximation scheme* (PTAS) for P is a family of algorithms $(A_\varepsilon)_{\varepsilon>0}$ such that, for any $\varepsilon > 0$, A_ε is a polynomial-time (randomized) $(1 + \varepsilon)$-approximation algorithm for P. An *efficient PTAS* (EPTAS) is a PTAS $(A_\varepsilon)_{\varepsilon>0}$ with running time of the form $f\left(\frac{1}{\varepsilon}\right) \cdot |I|^{O(1)}$, where f is an arbitrary computable function; In addition, $(A_\varepsilon)_{\varepsilon>0}$ is a (randomized) *fully PTAS* (FPTAS) if the running time of A_ε is of the form $\left(\frac{|I|}{\varepsilon}\right)^{O(1)}$.

2.3 Matroids and Membership Oracles

Definition 2.1. *A* matroid *is a set system $(E, \mathcal{I})$, where E is a finite set and $\mathcal{I} \subseteq 2^E, \mathcal{I} \neq \emptyset$ such that*

- *The* hereditary property*: for all $A \in \mathcal{I}$ and $B \subseteq A$ it holds that $B \in \mathcal{I}$.*
- *The* exchange property*: for all $A, B \in \mathcal{I}$ where $|A| > |B|$ there is $e \in A \setminus B$ such that $B + e \in \mathcal{I}$.*

An independent set $S \in \mathcal{I}$ is a basis *if $|S| = \max_{S' \in \mathcal{I}} |S'|$.*

In the common computational setting, the set system $(E, \mathcal{I})$ is not given explicitly as part of the input due to its large encoding size. The standard computational model that bypasses the issue is the *membership oracle* model; in this model, testing whether a given subset $S \subseteq E$ belongs to $\mathcal{I}$ is determined in a single query to the membership oracle. The oracle model enables to obtain unconditional information-theoretic based lower bounds for the running time of algorithms for matroid related problems. While these lower bounds hold in the oracle model, one may suspect that the lower bounds do not hold for any version of the matroid problem in which the matroid is encoded as part of the input. To rule this out, we also consider an alternative model in which the matroid $(E, \mathcal{I})$ is encoded as part of the input and membership can be decided in polynomial time. We further elaborate on this model in Section 5.

3 The Π-Matroid Family

In this section, we formally define the Π-matroid problem and prove an unconditional lower bound for solving it in the oracle model. In simple words, the matroids in this section are a family of matroids defined over the following parameters. The first parameter $n \in \mathbb{N}$ is the number of elements and the ground set is simply $[n]$. The second parameter, $k \in [n]$, is the rank of the

matroid.[1] Furthermore, α is a target value, which is usually equals to $\mathsf{sum}(S)$ for some $S \subseteq [n]$. Finally, the last parameter is a family of subsets $\Pi \subseteq 2^{[n]}$.

The set Π is called the *secret set* because finding Π is possible only via recurrent queries to the membership oracle of the matroid. Since Π can have an arbitrary structure, this may require a brute force enumeration. Generally, our use of the Π-matroid family is to reduce the hardness of finding $S \in \Pi$ to the problem of finding an independent set of the matroid with unique properties w.r.t. the parameters k, α, and Π.

Definition 3.1. *Let $n, k, \alpha \in \mathbb{N}_{>0}$. For some $\Pi \subseteq 2^{[n]}$, define the* **Π-matroid** *on n, k, α as $M_{n,k,\alpha}(\Pi) = ([n], \mathcal{I}_{n,k,\alpha}(\Pi))$, where*

$$\mathcal{I}_{n,k,\alpha}(\Pi) = \mathcal{J}_{n,k} \cup \mathcal{K}_{n,k,\alpha} \cup \mathcal{L}_{n,k,\alpha}(\Pi)$$

and $\mathcal{J}_{n,k}, \mathcal{K}_{n,k,\alpha}, \mathcal{L}_{n,k,\alpha}(\Pi)$ are defined as follows.

$$\begin{aligned} \mathcal{J}_{n,k} &= \big\{S \subseteq [n] \;\big|\; |S| < k\big\} \\ \mathcal{K}_{n,k,\alpha} &= \big\{S \subseteq [n] \;\big|\; |S| = k, \mathsf{sum}(S) \neq \alpha\big\} \\ \mathcal{L}_{n,k,\alpha}(\Pi) &= \big\{S \subseteq [n] \;\big|\; |S| = k, \mathsf{sum}(S) = \alpha, S \in \Pi\big\}. \end{aligned} \tag{1}$$

In simple words, $\mathcal{J}_{n,k}$ contains all subsets with strictly less than k elements; $\mathcal{K}_{n,k,\alpha}$ contains all subsets of cardinality k whose total sum is not α; finally, $\mathcal{L}_{n,k,\alpha}(\Pi)$ contains all subsets of cardinality k and total sum α which also belong to Π. Observe that $\mathcal{K}_{n,k,\alpha} \cup \mathcal{L}_{n,k,\alpha}(\Pi)$ is the set of bases of the matroid. See Figure 1 for an example of a member of the Π-matroid family. When clear from the context, we omit some/all of the parameters n, k, α. We first convince the reader that this is a family of matroids.

Lemma 3.2. *For every $n, k, \alpha \in \mathbb{N}_{>0}$, and $\Pi \subseteq 2^{[n]}$ it holds that $M(\Pi)$ is a matroid.*

Proof. First, note that $\emptyset \in \mathcal{J}$ since $0 < k$; therefore, $\emptyset \in \mathcal{I}(\Pi)$. For the hereditary property, let $A \in \mathcal{I}(\Pi)$. For all $B \subset A$ it holds that $|B| < k$; thus, $B \in \mathcal{J}$ and it follows that $B \in \mathcal{I}(\Pi)$. For the exchange property, let $A, B \in \mathcal{I}(\Pi)$ such that $|A| > |B|$. We consider the following cases.

1. $|B| < k - 1$. Then, for all $e \in A \setminus B$ it holds that $|B + e| < k - 1 + 1 = k$; thus, $B + e \in \mathcal{J}$ and it follows that $B + e \in \mathcal{I}(\Pi)$. Note that there is such $e \in A \setminus B$ because $|A| > |B|$.

2. $|B| = k - 1$ and $|A| = k$. We consider two sub cases.

   (a) $B \subseteq A$. Then, because $|A| > |B|$ there is $e \in A \setminus B$; thus, $B + e = A$ (because $|B| = k - 1$ and $|A| = k$). Hence, since $A \in \mathcal{I}(\Pi)$ it follows that $B + e \in \mathcal{I}(\Pi)$.

   (b) $B \not\subseteq A$. Then, since $|B| = k - 1$ and $|A| = k$ it follows that $|B \cap A| < |B| = k - 1$; thus, $|A \setminus B| = |A| - |A \cap B| > k - (k - 1) = 1$. Therefore, there are $e, f \in A \setminus B$ such that $e \neq f$. It follows that there is $g \in \{e, f\}$ such that $\mathsf{sum}(B + g) = \mathsf{sum}(B) + g \neq \alpha$. We conclude by (1) that $B + g \in \mathcal{K}$ and it follows that $B + g \in \mathcal{I}(\Pi)$ as required.

■

Using the Π-matroid family we define the following Π-*matroid problem*, where the goal is to decide if $\mathcal{L}(\Pi) \neq \emptyset$, where Π is some unknown fixed family of subsets of the ground set.

[1] The rank of the matroid is the maximum cardinality of an independent set of the matroid.

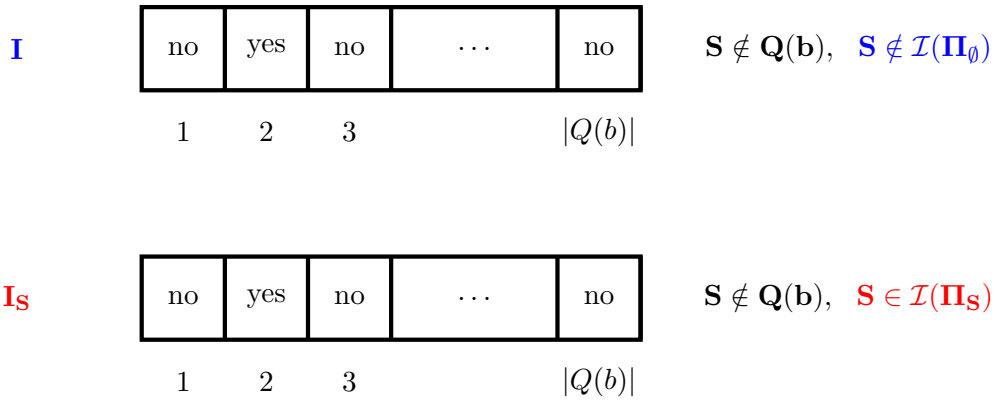


Figure 2: An illustration of the proof of Lemma 3.4. The figure presents the sequences of queries to the membership oracles by the algorithm on the instances I and I_S for some string of bits b such that $S \notin Q(b)$. The label "yes" ("no") means that the queried set is (not) independent in the matroid. The only query that distinguishes between I and I_S is on the set S, which is not queried; thus, the algorithm returns the same output for I and I_S.

Definition 3.3. *The* Π**-matroid problem***: Given* $n, k \in \mathbb{N}_{>0}$, $\alpha \in \left[n^2\right]$, *and a membership oracle to* $M_{n,k,\alpha}(\Pi)$, *for some* $\Pi \subseteq 2^{[n]}$, *decide if* $\mathcal{L}_{n,k,\alpha}(\Pi) \neq \emptyset$.

Observe that for some $n, k, \alpha \in \mathbb{N}_{>0}$, $\Pi \subseteq 2^{[n]}$, and $S \subseteq [n]$, if $S \in \mathcal{L}_{n,k,\alpha}(\Pi)$ then $S \in \Pi$ as well. Thus, by (1), finding an independent set S in the matroid with $|S| = k$ and $\mathsf{sum}(S) = \alpha$ is at least as hard as finding $S \in \Pi$. This gives the intuition for the hardness of the Π-matroid problem.

We use $I = (n, k, \alpha, M_{n,k,\alpha}(\Pi))$ to denote an instance of Π-matroid; we refer to n as the *size* of the instance I. A *solution* of I is a set $S \in \mathcal{L}(\Pi)$. It is important to note that the input in the Π-matroid problem does not contain a clear description of Π. Instead, finding Π requires a sequence of queries to the membership oracle of $M_{n,k,\alpha}(\Pi)$. The following hardness result relies on taking a family Π that consists of a single set. Roughly speaking, to decide such a Π-matroid instance $I = (n, k, \alpha, M_{n,k,\alpha}(\Pi))$, an algorithm for the problem must iterate over all (exponential many) subsets $S \subseteq [n]$ such that $|S| = k$ and $\mathsf{sum}(S) = \alpha$.

Lemma 3.4. *There is no randomized algorithm that decides the* Π*-matroid problem in time* $n^{O(1)}$, *where* n *is the size of the instance.*

Proof. Assume towards a contradiction that there is a randomized algorithm $\mathcal{A}$ that decides the Π-matroid problem in time $n^{O(1)}$, where n is the size of the instance size. Thus, there is a constant $c \in \mathbb{N}$ such that for every $n \in \mathbb{N}_{>0}$ and for every Π-matroid instance I of size n, algorithm $\mathcal{A}$ decides I in time bounded by n^c. Let $n \in \mathbb{N}_{>0}$ such that $\frac{2^n-1}{n^3} > 2n^c$; for simplicity, let $t = n^c$. For some $k, \alpha \in \mathbb{N}_{>0}$, consider the following set of all subsets of $[n]$ with cardinality k and sum exactly α:

$$\mathcal{F}_{n,k,\alpha} = \left\{S \subseteq [n] \;\middle|\; |S| = k, \mathsf{sum}(S) = \alpha\right\}.$$

Claim 3.5. *There are* $k \in [n]$ *and* $\alpha \in \left[n^2\right]$ *such that* $|\mathcal{F}_{n,k,\alpha}| \geq \frac{2^n-1}{n^3}$.

Proof. Observe that each $S \in 2^{[n]} \setminus \{\emptyset\}$ satisfies that $1 \leq |S| \leq n$ and that

$$1 \leq \mathsf{sum}(S) \leq |S| \cdot \max_{i \in [n]} i = n^2.$$

Thus, there are at most $n \cdot n^2 = n^3$ possibilities for choosing a pair of values (k, α), where $k = |S|$ and $\alpha = \mathsf{sum}(S)$ for some $S \in 2^{[n]} \setminus \{\emptyset\}$. Moreover, there are $2^n - 1$ non-empty subsets of $[n]$. Therefore, by the pigeonhole principle there is a pair of values $(k, \alpha) \in [n] \times \left[n^2\right]$ such that $|\mathcal{F}_{n,k,\alpha}| \geq \frac{2^n-1}{n^3}$. $\square$

Formally, given a Π-matroid instance $I = (n, k, \alpha, M_{n,k,\alpha}(\Pi))$, the randomized algorithm $\mathcal{A}$ produces a random string of bits $\bar{b} \in \{0,1\}^t$ and performs a sequence of queries to the membership oracle of $M_{n,k,\alpha}(\Pi)$ based on $\bar{b}, n, k, \alpha$, and the results of the previous queries; then, the algorithm decides the given instance based on the queries. Let $k \in [n]$ and $\alpha \in \left[n^2\right]$ such that $|\mathcal{F}_{n,k,\alpha}| \geq \frac{2^n-1}{n^3}$; the existence of these values is guaranteed by Claim 3.5. Let $\Pi_\emptyset = \emptyset$ and consider the Π-matroid instance $I = (n, k, \alpha, M_{n,k,\alpha}(\Pi_\emptyset))$. Given some string of bits $b \in \{0,1\}^t$, let $Q(b) \subseteq 2^{[n]}$ be the set of containing all $S \subseteq [n]$ queried by $\mathcal{A}$ on the instance I and string b (on the membership oracle of $M_{n,k,\alpha}(\Pi_\emptyset)$). For clarity, we use $\bar{b}$ for a random string and b for a realization of $\bar{b}$ to a specific string. Note that $Q(b)$ is a set since the algorithm is deterministic for every $b \in \{0,1\}^t$; conversely, $Q(\bar{b})$ is a random set for a random string $\bar{b} \in \{0,1\}^t$. Since the running time of algorithm $\mathcal{A}$ on a Π-matroid instance with n elements is bounded by t, it holds that $|Q(b)| \leq t$ for every $b \in \{0,1\}^t$. Let

$$R(I) = \left\{ S \in \mathcal{F}_{n,k,\alpha} \;\middle|\; \Pr\left(S \in Q(\bar{b})\right) \geq \frac{1}{2} \right\} \tag{2}$$

be all sets in $\mathcal{F}_{n,k,\alpha}$ that are queried by $\mathcal{A}$ w.p. at least $\frac{1}{2}$. We now upper bound $|R(I)|$:

$$\begin{aligned}
|R(I)| &= \left| \left\{ S \in \mathcal{F}_{n,k,\alpha} \;\middle|\; \Pr\left(S \in Q(\bar{b})\right) \geq \frac{1}{2} \right\} \right| \\
&\leq 2 \cdot \sum_{S \in \mathcal{F}_{n,k,\alpha}} \Pr\left(S \in Q(\bar{b})\right) \\
&= 2 \cdot \sum_{S \in \mathcal{F}_{n,k,\alpha}} \sum_{b \in \{0,1\}^t} \Pr\left(S \in Q(b)\right) \cdot \Pr(\bar{b} = b)
\end{aligned}$$

Thus, by changing the order of summation

$$\begin{aligned}
|R(I)| &\leq 2 \cdot \sum_{b \in \{0,1\}^t} \Pr(\bar{b} = b) \cdot \sum_{S \in \mathcal{F}_{n,k,\alpha} \cap Q(b)} \Pr\left(S \in Q(b)\right) \\
&\leq 2 \cdot \sum_{b \in \{0,1\}^t} \Pr(\bar{b} = b) \cdot |Q(b)| \\
&\leq 2t \cdot \sum_{b \in \{0,1\}^t} \Pr(\bar{b} = b) \\
&= 2 \cdot t \\
&< \frac{2^n - 1}{n^3} \\
&\leq |\mathcal{F}_{n,k,\alpha}|.
\end{aligned} \tag{3}$$

By (3) there is $S \in F_{n,k,\alpha} \setminus R(I)$; thus, $\Pr\left(S \in Q(\bar{b})\right) < \frac{1}{2}$. Now, consider the Π-matroid instance $I_S = (n, k, \alpha, M_{n,k,\alpha}(\Pi_S))$ such that $\Pi_S = \{S\}$. Observe that for all $b \in \{0,1\}^t$ such that $S \notin Q(b)$ it holds that the answers to all queries for subsets in $Q(b)$ are the same for both oracles (of $M_{n,k,\alpha}(\Pi_\emptyset)$ and $M_{n,k,\alpha}(\Pi_S)$); moreover, the decision which set to query next depends only on b, n, k, α, and the answers to previous queries. Hence, for all $b \in \{0,1\}^t$ such that $S \notin Q(b)$ algorithm $\mathcal{A}$ does not query the membership oracle of $M_{n,k,\alpha}(\Pi_S)$ with S. Therefore, w.p. strictly

larger than $\frac{1}{2}$ algorithm $\mathcal{A}$ does not query S for the instance I_S. Hence, w.p. strictly larger than $\frac{1}{2}$ algorithm $\mathcal{A}$ performs the exact same computations on I and I_S and must return the same result. We give an illustration in Figure 2. Since $\mathcal{L}_{n,k,\alpha}(\Pi_\emptyset) = \emptyset$ and $\mathcal{L}_{n,k,\alpha}(\Pi_S) \neq \emptyset$, we conclude that algorithm $\mathcal{A}$ cannot decide correctly w.p. at least $\frac{1}{2}$ both I and I_S, which is a contradiction that $\mathcal{A}$ is a randomized algorithm that decides every Π-matroid instance with size n in time bounded by n^c. ■

The following corollary follows directly from Lemma 3.4.

Corollary 3.6. *There is no deterministic polynomial algorithm for* Π*-matroid.*

4 Tight Lower Bounds using Π-Matroids

In this section we use the hardness of deciding the Π-matroid problem to give tight lower bound on the running time of randomized algorithms for several well studied weighted matroid problems. This includes the budgeted matroid independent set (intersection), exact weight matroid independent set, constrained minimum weight basis of a matroid, and knapsack cover with a a matroid (intersection). We aim to present the results in a compact manner; for this, we rely on the following generic results.

We show that an FPTAS for an optimization problem (either maximization or minimization) may be useful to derive an exact algorithm. Specifically, given a combinatorial optimization problem P, the family of *scaled* instances of P contains all instances I of P such that the following holds.

- The value of any solution for I w.r.t. P is a non negative integer.
- $\mathrm{OPT}(I) = |I|^{O(1)}$.

In addition, define the *P-scaled* optimization problem as: given a scaled instance I of P find a solution with optimum (either maximum or minimum, depends on P) value w.r.t. P. Clearly, if P is an optimization problem then so is P-scaled.

Lemma 4.1. *If a combinatorial optimization problem P admits a randomized* FPTAS*, then the P-scaled problem admits a randomized exact algorithm. Moreover, if P admits a deterministic* FPTAS *then P-scaled admits a deterministic exact algorithm.*

Proof. The proof for the randomized and deterministic versions are analogous and we illustrate the differences using parenthesizes in the appropriate places. Let P be a combinatorial optimization problem that admits a randomized (deterministic) FPTAS $\mathcal{A}$. By the definition of a scaled instance of P, there is $c \in \mathbb{N}$ such that for every scaled instance I of P it holds that $\mathrm{OPT}(I) \leq |I|^c$. Let I be a scaled instance I of P. In addition, let $\varepsilon = \frac{1}{2 \cdot |I|^c}$. Now, let S be the returned solution by $\mathcal{A}$ on I and the error parameter ε. Observe that the running time of the FPTAS $\mathcal{A}$ on I and ε is polynomial in $|I|$, by the definition of a randomized (deterministic) FPTAS and since $\varepsilon^{-1} = 2 \cdot |I|^c = |I|^{O(1)}$. Let $v(S)$ be the value of S w.r.t. the instance I of problem P. By the approximation guarantee of the randomized (deterministic) FPTAS $\mathcal{A}$ with our parameter ε it follows that w.p. at least $\frac{1}{2}$ (w.p. 1) it holds that

$$|v(S) - \mathrm{OPT}(I)| \leq \left| \max \left\{ \varepsilon \cdot \mathrm{OPT}(I), \left(\frac{\varepsilon}{1+\varepsilon} \right) \cdot \mathrm{OPT}(I) \right\} \right| \leq \varepsilon \cdot \mathrm{OPT}(I) \leq \frac{1}{2}. \tag{4}$$

The first inequality follows from the definition of an FPTAS, either for a minimization or maximization problem, respectively. The last inequality holds since $\mathrm{OPT}(I) \leq |I|^c$ and by the selection of ε. Since I is a scaled instance of P, the value of S is a non negative integer; thus, by (4) w.p. at least $\frac{1}{2}$ (w.p. 1) we construct a solution for I of value $v(S) = \mathrm{OPT}(I)$. Hence, we designed a randomized (deterministic) exact algorithm for the P-scaled problem. ■

The lower bounds we present in the following are based on the Π-matroid problem and are applied to matroid problems involving weights (e.g., profit, cost, etc.) and a budget (or, a cover demand, a target value). We are able to use in all of the constructions the same family of *standard weight* functions and *standard budgets* in slightly different ways. Specifically, for every $n \in \mathbb{N}_{>0}$ define the weight function of n as $w_n : [n] \to \mathbb{N}$ such that for all $i \in [n]$ it holds that

$$w_n(i) = i + 2 \cdot \mathsf{sum}([n]). \tag{5}$$

Moreover, define the standard budget of $n, k, \alpha \in \mathbb{N}_{>0}$ as

$$B_{n,k,\alpha} = \alpha + 2k \cdot \mathsf{sum}([n]). \tag{6}$$

We have the following general results that will be used in the next hardness results.

Lemma 4.2. *For every $n \in \mathbb{N}_{>0}$ and $S \subseteq [n]$, it holds that $w_n(S) \in \{0, 1, \ldots, 3 \cdot n^3\}$.*

Proof. By (5) we have $w_n(S) \in \mathbb{N}$. In addition, $0 = |\emptyset| \leq |S| \leq \mathsf{sum}(S) \leq w_n(S)$ and

$$w_n(S) \leq n \cdot \max_{i \in [n]} w_n(i) = n \cdot \max_{i \in [n]} i + n \cdot (2 \cdot \mathsf{sum}([n])) \leq n^2 + 2 \cdot n^3 \leq 3 \cdot n^3.$$

■

Lemma 4.3. *For every $n, k, \alpha \in \mathbb{N}_{>0}, \Pi \subseteq 2^{[n]}$, and $S \in \mathcal{L}_{n,k,\alpha}(\Pi)$, it holds that $w_n(S) = B_{n,k,\alpha}$.*

Proof. Let $S \in \mathcal{L}(\Pi)$. That is, by (1) it holds that $|S| = k$ and $\mathsf{sum}(S) = \alpha$. Then,

$$w_n(S) = \sum_{i \in S} (i + 2 \cdot \mathsf{sum}([n])) = \mathsf{sum}(S) + |S| \cdot 2 \cdot \mathsf{sum}([n]) = \alpha + 2k \cdot \mathsf{sum}([n]) = B_{n,k,\alpha}.$$

■

Lemma 4.4. *Let $n, k, \in \mathbb{N}_{>0}, \alpha \in \left[n^2\right], \Pi \subseteq 2^{[n]}$, and $S \in \mathcal{I}_{n,k,\alpha}(\Pi)$ such that $w_n(S) = B_{n,k,\alpha}$. Then, $S \in \mathcal{L}_{n,k,\alpha}(\Pi)$.*

Proof. By (1), we show that $|S| = k$ and $\mathsf{sum}(S) = \alpha$. First, Assume towards a contradiction that $|S| \neq k$. We consider two cases.

1. $|S| < k$. Then,

$$\begin{aligned} w_n(S) &= \sum_{i \in S} i + |S| \cdot 2 \cdot \mathsf{sum}([n]) \\ &\leq \sum_{i \in S} i + (k-1) \cdot 2 \cdot \mathsf{sum}([n]) \\ &\leq \mathsf{sum}([n]) + (k-1) \cdot 2 \cdot \mathsf{sum}([n]) \\ &= 2k \cdot \mathsf{sum}([n]) \\ &< B_{n,k,\alpha}. \end{aligned} \tag{7}$$

The last inequality holds since $\alpha \geq 1$.

2. $|S| > k$.

$$\begin{aligned} w_n(S) &= \sum_{i \in S} i + |S| \cdot 2 \cdot \mathsf{sum}([n]) \\ &\geq \sum_{i \in S} i + (k+1) \cdot 2 \cdot \mathsf{sum}([n]) \\ &\geq 2 \cdot \mathsf{sum}([n]) + 2k \cdot \mathsf{sum}([n]) \\ &> B_{n,k,\alpha}. \end{aligned} \tag{8}$$

The last inequality holds since: if $n = 1$ then $\alpha \leq 1 < 2 \cdot \mathsf{sum}([n])$; otherwise, $n > 1$ and it follows that $\alpha \leq n^2 < n^2 + n = 2 \cdot \mathsf{sum}([n])$.

By (7) and (8) we reach a contradiction that $w_n(S) = B_{n,k,\alpha}$. Thus, $|S| = k$. Therefore,

$$\mathsf{sum}(S) = \sum_{i \in S} i = w_n(S) - |S| \cdot 2 \cdot \mathsf{sum}([n]) = w_n(S) - 2k \cdot \mathsf{sum}([n]) = B_{n,k,\alpha} - 2k \cdot \mathsf{sum}([n]) = \alpha.$$

In conclusion, we have $S \in \mathcal{I}(\Pi)$, $|S| = k$ and $\mathsf{sum}(S) = \alpha$. Thus, by (1) it holds that $S \in \mathcal{L}(\Pi)$. ■

Corollary 4.5. *For every $n, k \in \mathbb{N}_{>0}$, $\alpha \in \left[n^2\right]$, and $\Pi \subseteq 2^{[n]}$ there is $S \in \mathcal{L}_{n,k,\alpha}(\Pi)$ if and only if there is $S \in \mathcal{I}_{n,k,\alpha}(\Pi)$ such that $w_n(S) = B_{n,k,\alpha}$.*

4.1 Budgeted Matroid Independent Set

Definition 4.6. Budgeted Matroid Independent Set (BM): *We are given a tuple $U = (E, \mathcal{I}, c, p, B)$, where $(E, \mathcal{I})$ is a matroid along with a membership oracle, $c : E \to \mathbb{R}_{\geq 0}$ is a cost function, $p : E \to \mathbb{R}_{\geq 0}$ is a profit function and $B \in \mathbb{R}_{\geq 0}$ is a budget. A solution of U is $S \subseteq E$ such that $c(S) \leq B$. The goal is to find a solution S of U with maximum profit $p(S)$.*

Lemma 4.7. *There is no randomized* FPTAS *for Budgeted Matroid Independent Set.*

Proof. We use a reduction from the Π-matroid problem. Let $I = (n, k, \alpha, M_{n,k,\alpha}(\Pi))$ be a Π-matroid instance. We define the BM instance $U_{n,k,\alpha}(\Pi) = ([n], \mathcal{I}_{n,k,\alpha}(\Pi), w_n, w_n, B_{n,k,\alpha})$ along with the membership oracle of $M_{n,k,\alpha}(\Pi)$; note that the standard weight function w_n is both the profit and the cost function, and that the budget is simply the standard budget with parameters n, k, α. Let $U = U_{n,k,\alpha}(\Pi)$ for simplicity. Note that U can be constructed from I in time $|I|^{O(1)}$.

By Lemma 4.2, for all $S \subseteq [n]$ it holds that $w_n(S) \in \mathbb{N}$ and that $\mathrm{OPT}(U) = n^{O(1)} = |U|^{O(1)}$; thus, U is a scaled instance of BM. Therefore, if BM has a randomized FPTAS, by Lemma 4.1 there is a randomized exact algorithm $\mathcal{A}$ that finds in time $|U|^{O(1)} = |I|^{O(1)}$ an optimal solution for U. Clearly, $\mathrm{OPT}(U) \leq B_{n,k,\alpha}$ since the profit and the cost of each element are the same. Therefore, by Corollary 4.5 the algorithm $\mathcal{A}$ returns a solution for U with profit $B_{n,k,\alpha}$ if and only if I has a solution. Hence, by computing $\mathcal{A}$ on U and getting a solution S, we can decide correctly I by returning that I has a solution if and only if $w_n(S) = B_{n,k,\alpha}$. Thus, we constructed a randomized polynomial (since $|U| = |I|^{O(1)}$) algorithm which decides the Π-matroid problem, contradicting Lemma 3.4.

■

Corollary 4.8. *There is no randomized* FPTAS *for Budgeted Matroid Intersection.*

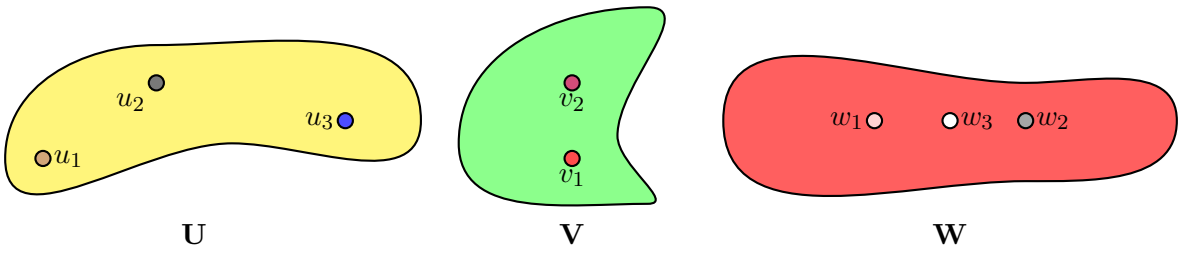


Figure 3: An example of a *partition matroid* $(E, \mathcal{I})$ - a matroid that can be efficiently encoded. The ground set is $E = \{u_1, u_2, u_3, v_1, v_2, w_1, w_2, w_3\}$, partitioned into three sets U, V, W. The independent sets are all subsets of E containing at most one element from U, V, and W; that is, $\mathcal{I} = \{S \subseteq E \mid \forall X \in \{U, V, W\} : |S \cap X| \leq 1\}$. A simple efficient encoding of $(E, \mathcal{I})$ is $I = (E, U, V, W)$. Membership can be decided efficiently given I by checking the feasibility of each set U, V, W.

4.2 Exact Weight Matroid Independent Set

Definition 4.9. exact matroid independent set (EMI): *We are given a tuple $I = (E, \mathcal{I}, w, T)$, where $(E, \mathcal{I})$ is a matroid along with a membership oracle, $w : E \to \mathbb{R}_{\geq 0}$ gives the elements weights, and $T \in \mathbb{R}_{\geq 0}$ is a target value. The goal is to find an independent set $S \in \mathcal{I}$ such that $w(S) = T$.*

We say that a problem is *Strongly* NP-Hard if it remains NP-Hard if all numbers in the instance of the problem are encoded in unary.

Lemma 4.10. EMI *is* Strongly NP-Hard.

Definition 4.11. Knapsack Cover with Matroid (KCM): *We are given a tuple $I = (E, \mathcal{I}, c, d, D)$, where $(E, \mathcal{I})$ is a matroid along with a membership oracle, $c : E \to \mathbb{R}_{\geq 0}$ is a cost function costs, $d : E \to \mathbb{R}_{\geq 0}$ is a size function, and $D \in \mathbb{R}_{>0}$ is a demand. A* solution *of I is an independent set $R \in \mathcal{I}$ such that $d(R) \geq D$.[2] The goal is to find a solution S of I of minimum cost $c(S)$.*

4.3 Constrained Minimum Basis of a Matroid

4.4 Knapsack Cover with a Matroid

4.5 Knapsack Cover with Matroid Intersection

5 The SAT-Matroid Family

The hardness result described in Section 3 shows unconditionally that the Π-matroid problem is very hard to solve. Nonetheless, this hardness result considers matroids with general membership oracles and does not give a lower bound for matroids that can be explicitly encoded. This is particularly important as matroids are often used to describe problems with an explicit encoding. This include partition matroids, graphic matroids, linear matroids, etc.[3] We formally define what we mean by explicitly encoded and define an encoding of matroids that do not rely on a membership oracle.

[2]For a technical reason, we define $D > 0$ so that any solution is not the empty set.

[3]See, e.g., [2] for a survey on various families of matroids.

Definition 5.1. *A function $f : \{0,1\}^* \to \mathbb{N} \times 2^{\mathbb{N}}$ is called* matroid encoder *if for every $I \in \{0,1\}^*$ it holds that $f(I) = \left(E_{f(I)}, \mathcal{I}_{f(I)}\right)$ is a matroid such that the following holds.*

1. *There is an algorithm that given $I \in \{0,1\}^*$ returns $E_{f(I)}$ in time $|I|^{O(1)}$.*
2. *There is an algorithm that given $I \in \{0,1\}^*$ and $S \subseteq E_{f(I)}$ decides if $S \in \mathcal{I}_{f(I)}$ in time $|I|^{O(1)}$.*

Note that every matroid has a matroid encoder; however, the encoding may be inefficient: a matroid $(E, \mathcal{I})$ encoded by $I \in \{0,1\}^*$ using a matroid encoder $f : \{0,1\}^* \to \mathbb{N} \times 2^{\mathbb{N}}$ (i.e., $f(I) = (E, \mathcal{I})$) may satisfy that $|I| = \Omega\left(2^{|E|}\right)$. This difficulty in general matroid representation has lead to the common use of membership oracles, which allow to analyze the complexity of algorithms for matroid problems considering only $|E|$ (and not the encoding of $\mathcal{I}$). This could have suggested that the hardness of the Π-matroid problem is rooted in the hardness that the membership oracle model manifest. Yet, there are matroids with a very efficient encoding. For examples, a *uniform matroid* $(E, \mathcal{I})$ such that $\mathcal{I} = \{S \subseteq E \mid |S| \leq k\}$ can be efficiently encoded using $I = (E, k) \in \{0,1\}^*$; clearly, deciding membership of some $S \subseteq E$ only depends on $|S|$ and k. Another example is given in Figure 3. We show that there are matroids that can be efficiently encoded (i.e., $|I| = |E|^{O(1)}$) but their corresponding Π-matroid problem is NP-Hard.

We use a reduction from the known SAT problem to a variation of the Π-matroid problem where the matroid is explicitly given by a matroid encoder. The main idea is to use an efficiently encodable member of the Π-matroid family, in which the secret set Π contains all feasible solutions of a given SAT instance. In a SAT instance $A = (V, \bar{V}, \mathcal{C})$ with $n \in \mathbb{N}$ variables (in a slightly simplified notation), we are given a set $V = \{v_1, \ldots, v_n\}$ of variables, their negations $\bar{V} = \{\bar{v}_1, \ldots, \bar{v}_n\}$, and a set $\mathcal{C} \subseteq 2^{V \cup \bar{V}}$ of clauses.[4] The goal is to decide if there is a set $S \subseteq [n]$ satisfying that for all $C \in \mathcal{C}$ there is $i \in [n]$ such that one of the following holds.

- $v_i \in C$ and $i \in S$.
- $\bar{v}_i \in C$ and $i \notin S$.

Such a set S is called a *solution* of A and let $\mathcal{S}(A)$ be the set of solutions of a SAT instance A. In addition, let $n(A) = n$ be the number of variables in the instance A. The family of SAT-matroids is the subfamily of Π-matroids where $\Pi = \mathcal{S}(A)$ for some SAT instance A.[5] Formally,

Definition 5.2. *Let A be a* SAT *instance, $k \in [n(A)]$, and $\alpha \in \left[n(A)^2\right]$. Define the* **SAT-matroid** *on A, k, α as $M_{n(A),k,\alpha}(\mathcal{S}(A)) = \left([n], \mathcal{I}_{n(A),k,\alpha}(\mathcal{S}(A))\right)$.*

We show below that the SAT-matroids can be efficiently encoded without the use of membership oracles. For every $I \in \{0,1\}^*$, we interpret I as $I = (A, k, \alpha)$ using standard unary encoding, where A is a SAT instance, $k \in [n(A)]$, and $\alpha \in \left[n(A)^2\right]$; w.l.o.g., we may assume that every $I \in \{0,1\}^*$ can be interpreted as such. Additionally, for simplicity we use $n_I = n(A), k_I = k, \alpha_I = \alpha$.

Definition 5.3. *Define the* **SAT-encoder** *function $f_{\text{SAT}} : \{0,1\}^* \to \mathbb{N} \times 2^{\mathbb{N}}$ such that for all $I \in \{0,1\}^*$ it holds that $f_{\text{SAT}}(I) = M_{n_I,k_I,\alpha_I}(\mathcal{S}(A_I))$.*

In the next result, we show that the SAT-encoder f_{SAT} is not only a matroid encoder, but it does so with an efficient encoding. This is crucial of reproducing a similar variant of the Π-matroid problem without a membership oracle involved.

[4] We can assume w.l.o.g. that for each clause C and $i \in [n]$ either $v_i \notin C$ or $\bar{v}_i \notin C$.

[5] Recall that Π-matroids are defined in Section 3.

Lemma 5.4. *The* SAT-encoder f_{SAT} *is a matroid encoder such that for every* $I \in \{0,1\}^*$ *it holds that* $|I| = |A_I|^{O(1)}$.

Proof. We first show that f_{SAT} is a matroid encoder. For all $I \in \{0,1\}^*$ it holds that $\mathcal{S}(A_I) \subseteq 2^{[n_I]}$; thus, by Lemma 3.2 it holds that f_{SAT} returns a matroid. We now prove that f_{SAT} satisfies the two conditions of Definition 5.1. First, given $I \in \{0,1\}^*$ we can compute $E_{f_{\text{SAT}}(I)} = [n_I]$ in time $|I|^{O(1)}$ by identifying the number of variables (i.e., n_I) in A. For the second condition of Definition 5.1, let $I \in \{0,1\}^*$; we show that we can decide membership in the matroid in efficient time. For every $S \subseteq [n_I]$ we can compute $|S|$ and $\mathsf{sum}(S)$ in time $|I|^{O(1)}$ as I is encoded in unary. Thus, validating if $S \in \mathcal{J}_{n_I,k_I}$ or $S \in \mathcal{K}_{n_I,k_I,\alpha_I}$ can be computed in time $|I|^{O(1)}$ regardless of $\mathcal{S}(A_I)$. In addition, deciding if $S \in \mathcal{S}(A_I)$ can be computed in time $|A_I|^{O(1)} = |I|^{O(1)}$ by validating that all clauses are satisfied. Thus, by (1) we can decide if $S \in \mathcal{L}_{n_I,k_I,\alpha_I}(\mathcal{S}(A_I))$. Overall, deciding whether $S \in \mathcal{J}_{n_I,k_I}$, $S \in \mathcal{K}_{n_I,k_I,\alpha_I}$, or $S \in \mathcal{L}_{n_I,k_I,\alpha_I}(\mathcal{S}(A_I))$ can be computed in time $|I|^{O(1)}$. By the above and (1), we can compute in time $|I|^{O(1)}$ correctly if a set S belongs to $\mathcal{I}_{f_{\text{SAT}}(I)} = \mathcal{I}_{n_I,k_I,\alpha_I}(\mathcal{S}(A_I))$. Thus, f_{SAT} is a matroid encoder by Definition 5.1. To conclude, observe that for all $I \in \{0,1\}^*$ it holds that $k_I, \alpha_I = {n_I}^{O(1)}$ and $n_I \leq |A|$ in unary encoding; thus, $|I| = |A|^{O(1)}$ as required. ■

The above lemma guarantees the efficient representation of SAT-matroids. This makes the following decision problem non-trivial to solve in polynomial time (in the encoding size of the instance). In fact, subsequently we show it is strongly NP-hard. For the following, recall that $\mathcal{L}_{n,k,\alpha}(\Pi)$ is defined in (1).

Definition 5.5. *The* **SAT-matroid problem:** *Given* $I \in \{0,1\}^*$*, decide if* $\mathcal{L}_{n_I,k_I,\alpha_I}(\mathcal{S}(A_I)) \neq \emptyset$.

For a SAT-matroid instance $I \in \{0,1\}^*$, A *solution* of I is a set $S \in \mathcal{L}_{n_I,k_I,\alpha_I}(\mathcal{S}(A_I))$.

Lemma 5.6. *For every computable function* $g : \mathbb{N} \to \mathbb{N}$*, if there is an algorithm that decides every* SAT*-matroid instance* $I \in \{0,1\}^*$ *in time* $O(g(|I|))$*, then there is an algorithm that decides every* SAT *instance* A *in time* $O\left(|A|^3 \cdot g\left(O\left(|A|^2\right)\right)\right)$.

Proof. Let $\mathcal{A}$ be an algorithm for SAT-matroid that decides every SAT-matroid instance $I \in \{0,1\}^*$ in time $O(g(|I|))$, where $g : \mathbb{N} \to \mathbb{N}$ is some computable function. Let A be a SAT instance and let $n = n(A)$. We describe the following algorithm which decides A. First, check if $\emptyset \in \mathcal{S}(A)$. Then, for every $k \in [n]$ and $\alpha \in \left[n^2\right]$, compute algorithm $\mathcal{A}$ on the SAT-matroid $I_{A,k,\alpha} = (A, k, \alpha)$. For every $k \in [n]$ and $\alpha \in \left[n^2\right]$, observe that a solution $S \in \mathcal{L}_{n,k,\alpha}(\mathcal{S}(A))$ for the SAT-matroid instance $I_{A,k,\alpha}$ is in particular a solution $S \in \mathcal{S}(A)$ for A. Thus, if there are $k \in [n]$ and $\alpha \in \left[n^2\right]$ such that algorithm $\mathcal{A}$ on instance $I_{A,k,\alpha}$ decides that $\mathcal{L}_{n,k,\alpha}(\mathcal{S}(A)) \neq \emptyset$, or that $\emptyset \in \mathcal{S}(A)$, we return that A has a solution; otherwise, we return that A does not have a solution. To show that the above algorithm correctly decides A (and the SAT problem), consider the following cases.

1. The instance A has a solution $S \in \mathcal{S}(A)$. If $S = \emptyset$ we necessarily decide the instance correctly. Otherwise, let $k = |S|$ and $\alpha = \mathsf{sum}(S)$. By (1) it follows that the SAT-matroid instance $I_{A,k,\alpha}$ has a solution, which is S. Therefore, when algorithm $\mathcal{A}$ decides instance $I_{A,k,\alpha}$, it must decide that $\mathcal{L}_{n,k,\alpha}(\mathcal{S}(A)) \neq \emptyset$

2. The instance A does not have a solution, i.e., $\mathcal{S}(A) = \emptyset$. Clearly, $\emptyset \notin \mathcal{S}(A)$ here. To conclude, assume towards a contradiction that there are $k \in [n]$ and $\alpha \in \left[n^2\right]$ such that there is a solution S for $I_{A,k,\alpha}$. Thus, $S \in \mathcal{L}_{n,k,\alpha}(\mathcal{S}(A))$ by Definition 5.5; therefore, since $\mathcal{L}_{n,k,\alpha}(\mathcal{S}(A)) \subseteq \mathcal{S}(A)$ it follows that $S \in \mathcal{S}(A)$ in contradiction that $\mathcal{S}(A) = \emptyset$.

Observe that $n \leq |A|$ as A contains n distinct variables; in addition, observe that $|I_{A,k,\alpha}| = |A| + k + \alpha \leq |A| + n + n^2 = O(|A|^2)$. Thus, the number of iterations (i.e., distinct options for the parameters k, α) is bounded by $O(|A|^3)$, and the running time of each iteration is bounded by $O(g(O(|A|^2)))$. Therefore, our algorithm decides SAT in time $O(|A|^3 \cdot g(O(|A|^2)))$. ■

The following corollary is a consequence of Lemma 5.6.

Corollary 5.7. *The* SAT*-matroid problem is* NP-Hard*.*

Proof. By Lemma 5.6 if there is a polynomial time algorithm that decides the SAT-matroid problem, there is a polynomial time algorithm that decides SAT. Since SAT is well-known to be NP-Hard [1], the proof follows. ■

6 Tight Lower Bounds using SAT-Matroids

6.1 Budgeted Matroid Independent Set

We define the variation of BM where the given matroid is explicitly encoded (see Definition 5.1).

Definition 6.1. *Given a matroid encoder $f : \{0,1\}^* \to \mathbb{N} \times 2^{\mathbb{N}}$, define the* **$f$-budgeted matroid independent set (f-BM)** *problem as follows. An instance of f-BM is a tuple $U = (I, c, p, B)$, where $I \in \{0,1\}^*$, $c : E_{f(I)} \to \mathbb{R}_{\geq 0}$ is a cost function, $p : E_{f(I)} \to \mathbb{R}_{\geq 0}$ is a profit function and $B \in \mathbb{R}_{\geq 0}$ is a budget. A solution of U is $S \in \mathcal{I}_{f(I)}$ such that $c(S) \leq B$. The goal is to find a solution S of U with maximum profit $p(S)$.*

Lemma 6.2. *Unless* P=NP*, there is no* FPTAS *for* f_{SAT}-BM*.*

Proof. We use a reduction from the SAT-matroid problem. Let $I \in \{0,1\}^*$ be a SAT-matroid instance; for simplicity, let $A = A_I, n = n(A), k = k_I$, and $\alpha = \alpha_I$. We define the f_{SAT}-BM instance

$$U = (I, w_n, w_n, B_{n,k,\alpha}).$$

Note that $f_{\text{SAT}}(I) = M_{n,k,\alpha}(\mathcal{S}(A))$ by Definition 5.3. Moreover, by Lemma 5.4, (5), and (6) it follows that U can be constructed from I in time $|I|^{O(1)}$. To show that U is a scaled f_{SAT}-BM instance, observe that for all $S \subseteq [n]$ it holds that $w_n(S) \in \mathbb{N}$ and that $\text{OPT}(U) = |U|^{O(1)}$ by Lemma 4.2. Therefore, if f_{SAT}-BM has an FPTAS, by Lemma 4.1 there is an exact algorithm $\mathcal{A}$ that finds in time $|U|^{O(1)} = |I|^{O(1)}$ an optimal solution for U. Clearly, $\text{OPT}(U) \leq B_{n,k,\alpha}$ since the profit and the cost of each element are the same. Therefore, by Corollary 4.5 the algorithm $\mathcal{A}$ on instance U returns a solution for U with profit $B_{n,k,\alpha}$ if and only if I has a solution. Hence, by computing $\mathcal{A}$ on U and getting a solution S, we can decide correctly I by returning that I has a solution if and only if $w_n(S) = B_{n,k,\alpha}$. The above algorithm decides the SAT-matroid problem, which is impossible by Corollary 5.7, unless P=NP. ■

7 Discussion

References

[1] Cook, S.A.: The complexity of theorem-proving procedures. In: Logic, Automata, and Computational Complexity: The Works of Stephen A. Cook, pp. 143–152 (2023)

[2] Schrijver, A., et al.: Combinatorial optimization: polyhedra and efficiency, vol. 24. Springer (2003)